

HUNDRED DAYS OF CODE

DAY 7

Question 1:

Write a C program to find time of flight of a projectile when initial velocity, angle, and gravity are given.

STEP 1 – REQUIRED INPUT, OUTPUT, AND FORMULA

We are given: Initial velocity = u

Angle of projection = θ

Acceleration due to gravity = g

We need to calculate time of flight T

Formula

$$T = 2u \sin \theta / g$$

will need the $\sin()$ function.

$\sin()$ in C uses **radians**; radians = $\theta \times \pi / 180$

Input

1. Initial velocity u
2. Angle theta
3. Gravity g

Output

Time of flight T in seconds.

STEP 2 – TEST CASE TABLE

TEST CASE	INPUT	EXPECTED OUTPUT	ACTUAL OUTPUT	RESULT
1	$u = 20 \text{ m/s}$, $\theta = 30^\circ$, $g = 9.8 \text{ m/s}^2$	2.04 s	2.04 s	pass
2	$u = 30 \text{ m/s}$, $\theta = 45^\circ$, $g = 9.8 \text{ m/s}^2$	4.33 s	4.33 s	pass
3	$u = 10 \text{ m/s}$, $\theta = 60^\circ$, $g = 9.8 \text{ m/s}^2$	1.77 s	1.77 s	pass

HUNDRED DAYS OF CODE

DAY 7

STEP 3 – ALGORITHM

Algorithm — Time of Flight

1. Start.
2. Declare variables u, theta, g, radian and T.
3. Read the initial velocity u.
4. Read the angle theta.
5. Read the gravitational acceleration g.
6. Convert theta from degrees to radians.
7. Calculate time of flight using
$$T = \frac{2u\sin(\theta)}{g}.$$
8. Display the time of flight.
9. Stop.

EVALUATION OF ALGORITHM

Test Case 1:

Given:

- $u = 20$
- $\theta = 30^\circ$
- $g = 9.8$

Convert:

$$30 \times \frac{\pi}{180} = 0.5236$$

$$T = \frac{2(20)\sin(0.5236)}{9.8}$$
$$T \approx 2.04s$$

Expected output = 2.04 seconds

Test Case 2

Given:

- Initial velocity, $u = 30\text{m/s}$
- Angle, $\theta = 45^\circ$
- Gravity, $g = 9.8\text{m/s}^2$

HUNDRED DAYS OF CODE

DAY 7

$$\theta = 45 \times \frac{\pi}{180}$$
$$\theta = 0.7854 \text{ radians}$$

$$T = \frac{2(30)\sin(0.7854)}{9.8}$$

$$\sin(45^\circ) \approx 0.7071$$

$$T = \frac{60(0.7071)}{9.8}$$

$$T = \frac{42.426}{9.8}$$

$$T \approx 4.33 \text{ seconds}$$

Expected output = 4.33 seconds

Test Case 3

Given:

- Initial velocity, $u = 10\text{m/s}$
- Angle, $\theta = 60^\circ$
- Gravity, $g = 9.8\text{m/s}^2$

$$\theta = 60 \times \frac{\pi}{180}$$
$$\theta = 1.0472 \text{ radians}$$

$$T = \frac{2(10)\sin(1.0472)}{9.8}$$

$$\sin(60^\circ) \approx 0.8660$$

$$T = \frac{20(0.8660)}{9.8}$$

$$T = \frac{17.32}{9.8}$$

$$T \approx 1.77 \text{ seconds}$$

HUNDRED DAYS OF CODE

DAY 7

Expected output = 1.77 seconds

STEP 4 – REQUIRED C PROGRAM

```
#include <stdio.h>

#include <math.h>

int main()

{

    float u, theta, g, radian, T;

    printf("Enter initial velocity (m/s): ");

    scanf("%f", &u);

    printf("Enter angle of projection (degrees): ");

    scanf("%f", &theta);

    printf("Enter acceleration due to gravity (m/s^2): ");

    scanf("%f", &g);

    radian = theta * 3.14159 / 180.0;

    T = (2 * u * sin(radian)) / g;

    printf("Time of flight = %.2f seconds\n", T);

    return 0;

}
```

HUNDRED DAYS OF CODE

DAY 7

STEP 5 – COMPILATION AND EXECUTION

```
(kali㉿kali)-[~]  
$ vi q1.c  
  
(kali㉿kali)-[~]  
$ gcc q1.c -o q1 -lm  
  
(kali㉿kali)-[~]  
$ ./q1  
Enter initial velocity (m/s): 20  
Enter angle of projection (degrees): 30  
Enter acceleration due to gravity (m/s^2): 9.8  
Time of flight = 2.04 seconds  
  
(kali㉿kali)-[~]  
$ ./q1  
Enter initial velocity (m/s): 30  
Enter angle of projection (degrees): 45  
Enter acceleration due to gravity (m/s^2): 9.8  
Time of flight = 4.33 seconds  
  
(kali㉿kali)-[~]  
$ ./q1  
Enter initial velocity (m/s): 10  
Enter angle of projection (degrees): 60  
Enter acceleration due to gravity (m/s^2): 9.8  
Time of flight = 1.77 seconds  
  
(kali㉿kali)-[~]  
$
```

QUESTION 2: Write a C program to find mobility when drift velocity and electric field are given.

STEP 1: REQUIRED INPUT, OUTPUT and FORMULA

We are given:

- Drift velocity v_d
- Electric field E

We need to calculate **mobility** μ .

HUNDRED DAYS OF CODE

DAY 7

Formula used:

$$\mu = \frac{v_d}{E}$$

Input

- Drift velocity
- Electric field

Output

- Mobility

The SI unit of mobility is: $\text{m}^2 / (\text{V} \cdot \text{s})$

STEP 2- TEST CASE TABLE

TEST CASE	INPUT	EXPECTED OUTPUT	ACTUAL OUTPUT	RESULT
1	$v_d=10\text{m/s}$, $E = 5\text{V/m}$	$2.00 \text{ m}^2/(\text{V} \cdot \text{s})$	2.00	PASS
2	$v_d=20\text{m/s}$, $E = 4\text{V/m}$	$5.00 \text{ m}^2/(\text{V} \cdot \text{s})$	5.00	PASS
3	$v_d = 15\text{m/s}$, $E = 3\text{V/m}$	$5.00 \text{ m}^2/(\text{V} \cdot \text{s})$	5.00	PASS

Step 3 — Algorithm

Algorithm — Mobility

1. Start.
2. Declare variables v_d , E and mobility.
3. Read the drift velocity v_d .
4. Read the electric field E .
5. Calculate mobility using $\mu = v_d/E$.
6. Display the mobility.
7. Stop.

Evaluate the algorithm

For Test Case 1:

$$\begin{aligned} v_d &= 10 \\ E &= 5 \end{aligned}$$

HUNDRED DAYS OF CODE

DAY 7

Therefore: $\mu = \frac{10}{5} = 2$

Expected result: **2.00 m²/(V·s)**

Test Case 2

Given:

- Drift velocity, $v_d = 20\text{m/s}$
- Electric field, $E = 4\text{V/m}$

$$\mu = \frac{20}{4}$$

$\mu = 5.00 \text{ m}^2/(\text{V} \cdot \text{s})$
--

Expected output = 5.00 m²/(V·s)

Test Case 3

Given:

- Drift velocity, $v_d = 15\text{m/s}$
- Electric field, $E = 3\text{V/m}$

$$\mu = \frac{15}{3}$$

$\mu = 5.00 \text{ m}^2/(\text{V} \cdot \text{s})$
--

Expected output = 5.00 m²/(V·s)

STEP 4 - REQUIRED C PROGRAM

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    float vd, E, mobility;
```

HUNDRED DAYS OF CODE

DAY 7

```
printf("Enter drift velocity (m/s): ");

scanf("%f", &vd);

printf("Enter electric field (V/m): ");

scanf("%f", &E);

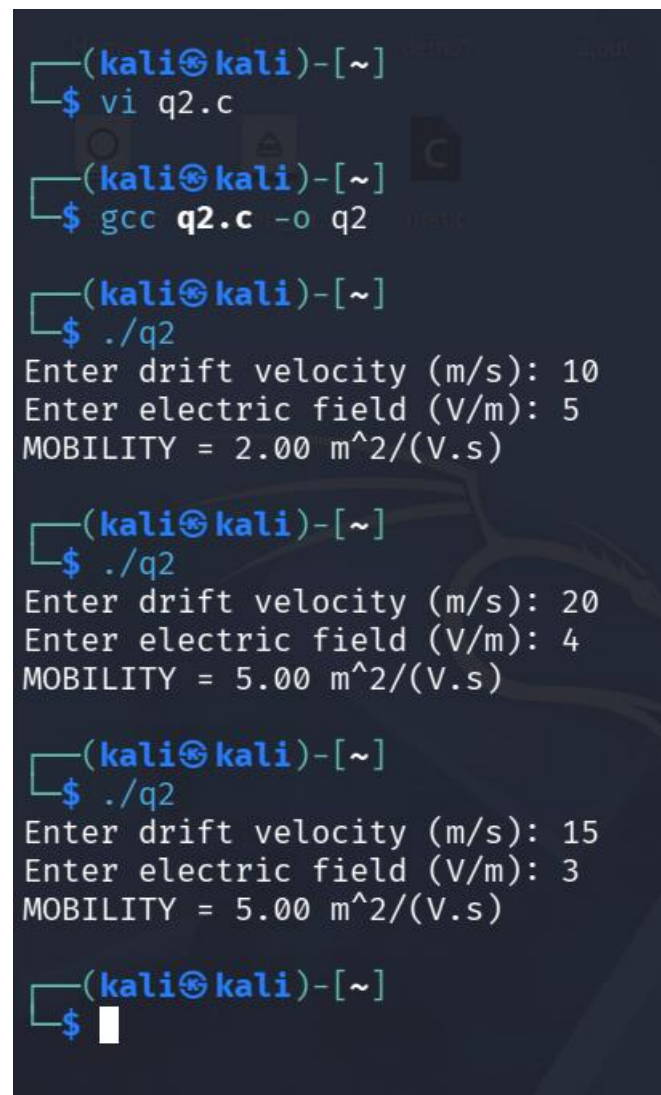
mobility = vd / E;

printf("Mobility = %.2f m^2/(V.s)\n", mobility);

return 0;

}
```

STEP 5 – COMPILATION AND EXECUTION



```
(kali㉿kali)-[~]
└─$ vi q2.c

(kali㉿kali)-[~]
└─$ gcc q2.c -o q2

(kali㉿kali)-[~]
└─$ ./q2
Enter drift velocity (m/s): 10
Enter electric field (V/m): 5
MOBILITY = 2.00 m^2/(V.s)

(kali㉿kali)-[~]
└─$ ./q2
Enter drift velocity (m/s): 20
Enter electric field (V/m): 4
MOBILITY = 5.00 m^2/(V.s)

(kali㉿kali)-[~]
└─$ ./q2
Enter drift velocity (m/s): 15
Enter electric field (V/m): 3
MOBILITY = 5.00 m^2/(V.s)

(kali㉿kali)-[~]
└─$
```


HUNDRED DAYS OF CODE

DAY 7

Question 3:

Write a C program to find work function when photon energy and maximum kinetic energy are given.

STEP 1 – REQUIRED INPUT, OUTPUT , and FORMULA

We are given:

- Photon energy E_{photon}
- Maximum kinetic energy K_{max}

need to calculate the **work function** ϕ .

Formula

$$\phi = E_{photon} - K_{max}$$

therefore , **Work function = Photon energy – Maximum kinetic energy**

STEP 2 – TEST CASE TABLE

TEST CASE	INPUT	EXPECTED OUTPUT	ACTUAL OUTPUT	RESULT
1	Photon energy = 5 eV, KE = 2 eV	3.00eV	3.00eV	PASS
2	Photon energy = 8 eV, KE = 3 eV	5.00eV	5.00eV	PASS
3	Photon energy = 10 eV, KE = 4 eV	6.00eV	6.00eV	PASS

Step 3 — Algorithm

Algorithm — Work Function

1. Start.
2. Declare variables photonEnergy, kineticEnergy and workFunction.
3. Read the photon energy.
4. Read the maximum kinetic energy.
5. Calculate work function using
 $\phi = E_{photon} - K_{max}$.
6. Display the work function.

HUNDRED DAYS OF CODE

DAY 7

7. Stop.

EVALUATION OF THE ALGORITHM:

Test Case 1

Given:

$$E_{\text{photon}} = 5\text{eV}$$
$$K_{\text{max}} = 2\text{eV}$$

Therefore:

$$\phi = 5 - 2 = 3\text{eV}$$

Expected output: **3.00 Ev**

Test Case 2

Given:

- Photon energy = 8 eV
- Maximum kinetic energy = 3 eV

$$\phi = 8 - 3$$
$$\boxed{\phi = 5.00\text{ eV}}$$

Expected output = **5.00 eV**

Test Case 3

Given:

- Photon energy = 10 eV
- Maximum kinetic energy = 4 eV

$$\phi = 10 - 4$$
$$\boxed{\phi = 6.00\text{ eV}}$$

Expected output = **6.00 eV**

HUNDRED DAYS OF CODE

DAY 7

STEP 4 – REQUIRED C PROGRAM

```
#include <stdio.h>

int main()
{
    float photonEnergy, kineticEnergy, workFunction;

    printf("Enter photon energy (eV): ");
    scanf("%f", &photonEnergy);

    printf("Enter maximum kinetic energy (eV): ");
    scanf("%f", &kineticEnergy);

    workFunction = photonEnergy - kineticEnergy;

    printf("Work function = %.2f eV\n", workFunction);

    return 0;
}
```

HUNDRED DAYS OF CODE

DAY 7

STEP 5 – COMPILATION AND EXECUTION

```
(kali㉿kali)-[~]
$ vi q3.c

(kali㉿kali)-[~]
$ gcc q3.c -o q3

(kali㉿kali)-[~]
$ ./q3
Enter photon energy (eV): 5
Enter maximum kinetic energy (eV): 2
Work function = 3.00 eV

(kali㉿kali)-[~]
$ ./q3
Enter photon energy (eV): 8
Enter maximum kinetic energy (eV): 3
Work function = 5.00 eV

(kali㉿kali)-[~]
$ ./q3
Enter photon energy (eV): 10
Enter maximum kinetic energy (eV): 4
Work function = 6.00 eV

(kali㉿kali)-[~]
$
```

Result

The three C programs were successfully designed, compiled, and executed.

The algorithms were evaluated using three test cases for each problem, and the calculated expected outputs were obtained correctly. The C programs were then compiled and executed, and the actual outputs matched the expected outputs.

Therefore, all three programs passed the given test cases successfully.

.....

HUNDRED DAYS OF CODE
DAY 7

NAME – AYUSHI KUMAWAT

SAP ID - 590032252

BATCH – 16

COURSE – B.TECH CSE 2026-2030

.....

THANKYOU